Redrob Candidate Ranker

India Runs: Data & AI Challenge

Team Name: Data Enthusiast

Team Leader: Praasuk Jain

Problem Statement: Recruiters miss perfect candidates because legacy systems rely on rigid keyword matching instead of understanding true fit and availability.

Solution Overview

What is your proposed solution?

An advanced 5-stage intelligent pipeline powered by a Two-Stage NLP architecture (Bi-Encoder + Cross-Encoder) and a Latent Skill Inference engine.

What differentiates your approach?

1. **Two-Stage NLP Pipeline:** We run a lightning-fast Bi-Encoder to find the Top 1,000, then apply an ultra-accurate **Cross-Encoder** (`ms-marco`) exclusively to the Top 200, achieving state-of-the-art accuracy locally on CPU.
2. **Latent Skill Inference:** We mathematically deduce missing foundational skills (e.g., Candidates who list `Langchain` are automatically credited for `Python` and `Generative AI`).
3. **Lifelong Learner Index:** We automatically reward candidates who demonstrate continuous upskilling.

JD Understanding & Candidate Evaluation

What are the key requirements extracted?

Core competencies (embeddings, retrieval, python), product-company experience (vs pure service), and minimum 3 years of experience.

Which candidate signals are most important?

Beyond skills, we heavily weigh behavioral signals: `recruiter_response_rate`, `notice_period_days`, and `career_stability_index`. A candidate who matches perfectly but never responds or jumps jobs every 3 months is heavily penalized, reflecting real-world recruiter priorities.

Ranking Methodology

How does your system retrieve, score, and rank?

1. **Hard Filter:** Drops candidates with `<3` YoE or honeypot traps.
2. **Scoring:** BM25 (with Latent Skill Inference) mixed with Behavioral & Learner Multipliers.
3. **Bi-Encoder Re-Ranking:** The Top 1000 are scored via Cosine Similarity (`all-MiniLM-L6-v2`).
4. **Cross-Encoder Re-Ranking:** The Top 200 are run through a deep-interaction Cross-Encoder (`ms-marco-MiniLM-L-2-v2`).
5. **Aggregation:** Reciprocal Rank Fusion perfectly combines the separate discrete ranks.

Explainability & Data Validation

How are ranking decisions explained?

We wrote a Hyper-Personalized Extraction algorithm that dynamically reads the candidate's exact work history, finds the exact sentence that made them a match, and writes a personalized justification.

(Example Output: "5 yrs exp. Highly relevant: built 'real-time recommendation engine' at Company X. Highly responsive (100%). Lifelong Learner.")

How do you prevent hallucinations?

No generative LLMs are used for the reasoning. The explanations are deterministically extracted from verifiable data points within the candidate's JSON profile.

End-to-End Workflow

1. **Ingest:** Stream `candidates.jsonl` (handles massive files efficiently).
2. **Filter & Infer:** Instantly discard honeypots and infer missing latent skills.
3. **Expand & Retrieve:** Run BM25 with AI Query Expansion to pull the Top 1,000 matches.
4. **Bi-Encoder:** Convert candidate summaries into dense vectors to shrink pool to 200.
5. **Cross-Encoder:** Perform deep contextual scoring on the final 200.
6. **RRF Aggregation:** Compute `1 / (k + rank)` to finalize the Top 100.
7. **Output:** Generate `team_PJ2001_IND.csv` with dynamically generated context.

System Architecture

- **Data Layer:** Streaming JSONL parser.
- **Retrieval Engine:** Custom BM25 implementation with Latent Skill Inference.
- **NLP Engine (Stage 1):** HuggingFace Bi-Encoder (`all-MiniLM-L6-v2`) on CPU.
- **NLP Engine (Stage 2):** HuggingFace Cross-Encoder (`ms-marco`) on CPU.
- **Aggregation Layer:** Reciprocal Rank Fusion (RRF) engine.
- **Output Layer:** CSV writer with dynamic context extraction.

(Everything runs 100% locally. No external API calls are made.)

Results & Performance

What results demonstrate ranking quality?

The system successfully prioritized candidates who described building "recommendation engines" or "vector DBs" conceptually, even if they didn't explicitly list the exact keywords requested in the JD, proving deep semantic understanding.

How does it meet compute constraints?

- **Scale:** Successfully evaluated the entire 100,000 candidate dataset.
- **Speed:** Total end-to-end runtime of **47.62 seconds** on a standard CPU, proving that true Cross-Encoder architectures are possible when properly staged.
- **Memory:** Peaked well below the 16GB limit due to lazy-loading and stream processing.

Technologies Used

- **Python (3.11+):** Core pipeline execution.
- **Sentence-Transformers:** Used both `all-MiniLM-L6-v2` for rapid wide-scale retrieval and `cross-encoder/ms-marco-MiniLM-L-2-v2` for hyper-accurate narrow-scale re-ranking.
- **Scikit-Learn:** For computing Cosine Similarity arrays efficiently.
- **Streamlit:** For building the interactive sandbox web interface.
- **Hugging Face Spaces:** Used to host the live Sandbox environment.

Submission Assets

- **Live Sandbox Demo:** https://huggingface.co/spaces/praasukjain2001/redrob-ranker
- **GitHub Repository:** https://github.com/PJ2001-IND/redrob-ranker
- **Output File:** `team_PJ2001_IND.csv`
- **Metadata:** `submission_metadata.yaml`

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU